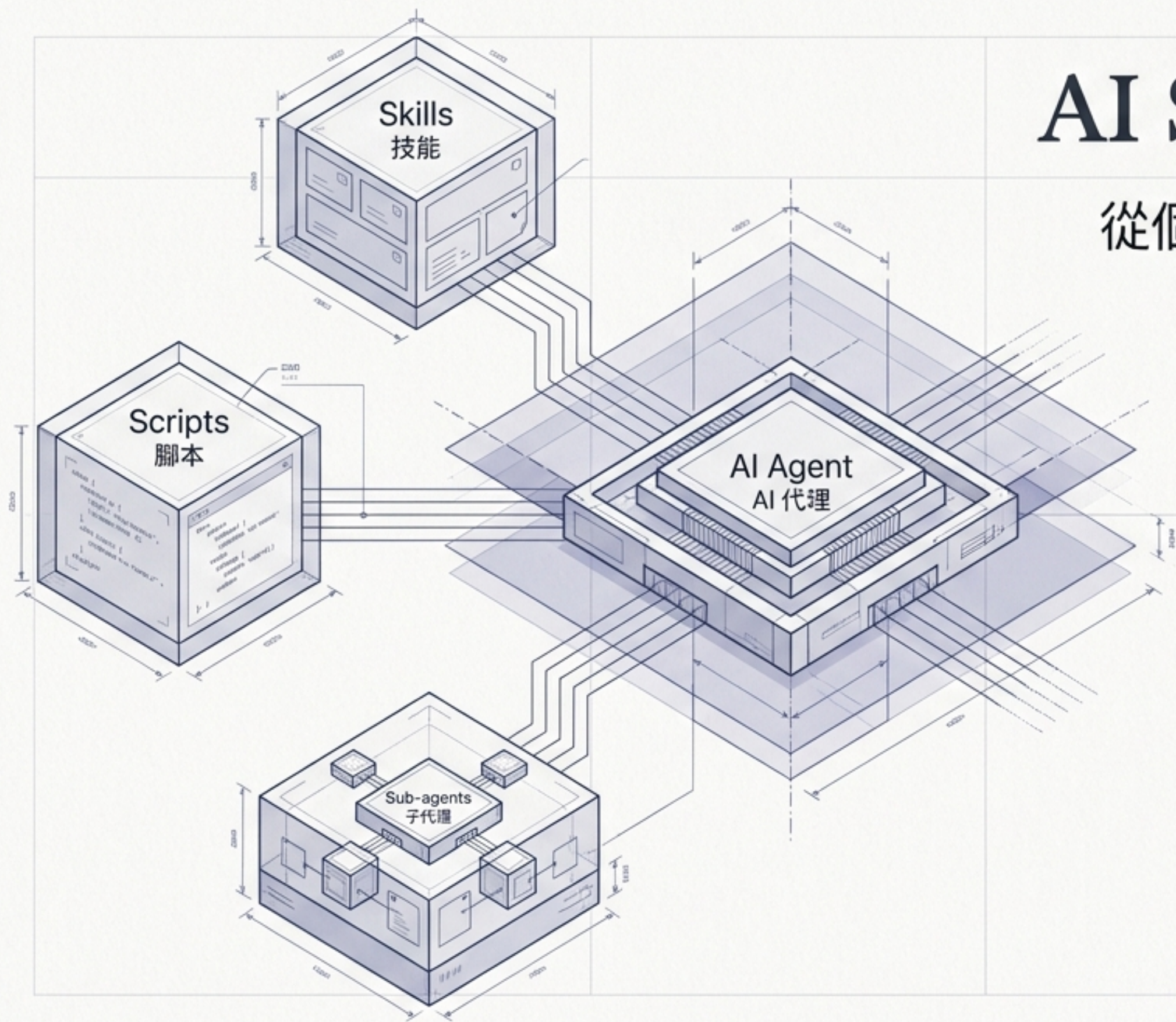


AI Skill 實戰架構藍圖

從個人生產力到團隊資產：打造穩定、自動觸發且高維護性的 AI 技能



- ✓ 核心心法 (Mindsets for AI PMs)
- ✓ 實戰場景 (Real-World Architectures)
- ✓ 長效維護 (Long-Term Compounding)

寫 Skill 不難，難的是「長效穩定」

表象挑戰

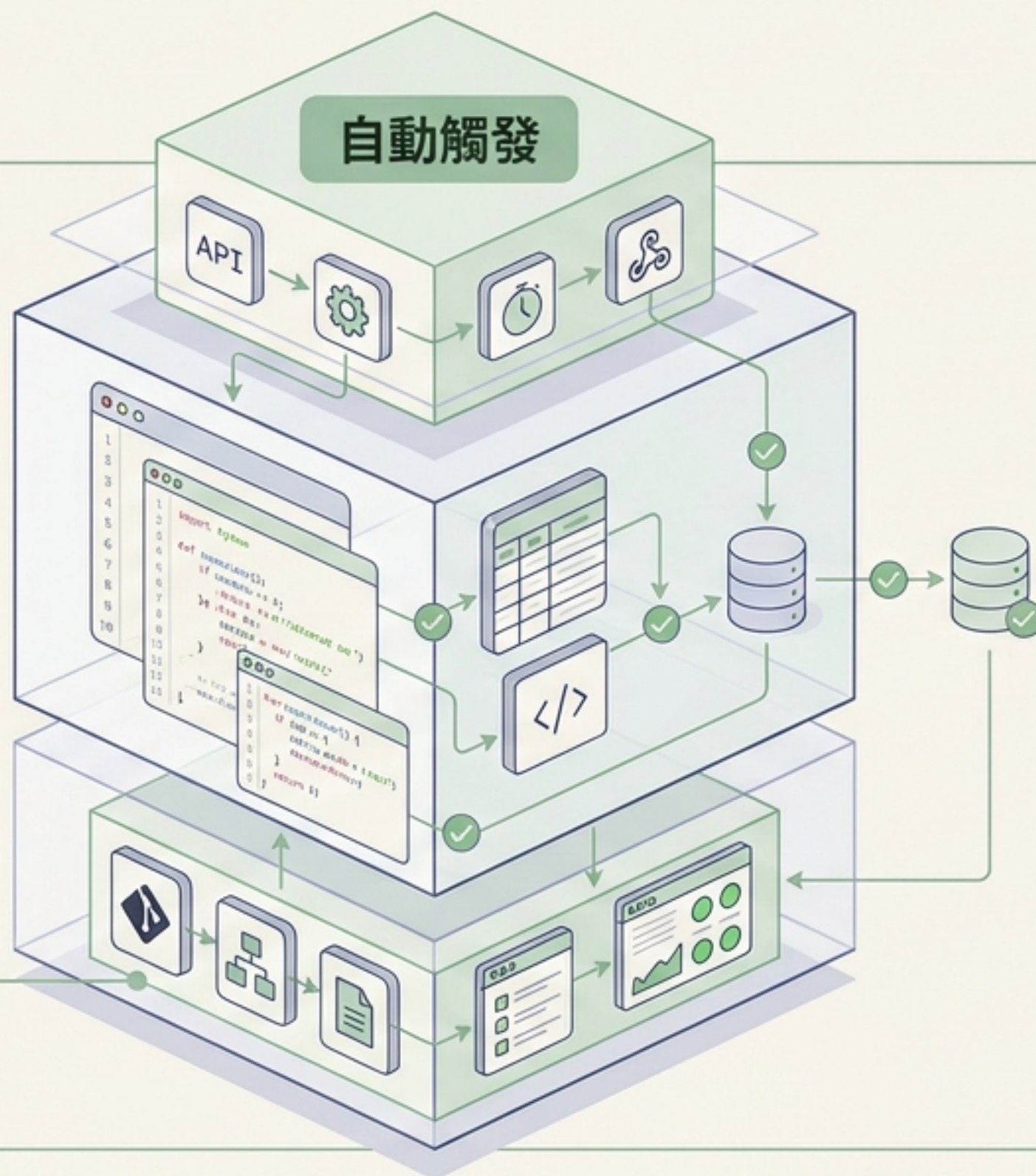
撰寫冗長的 Prompt



真實挑戰

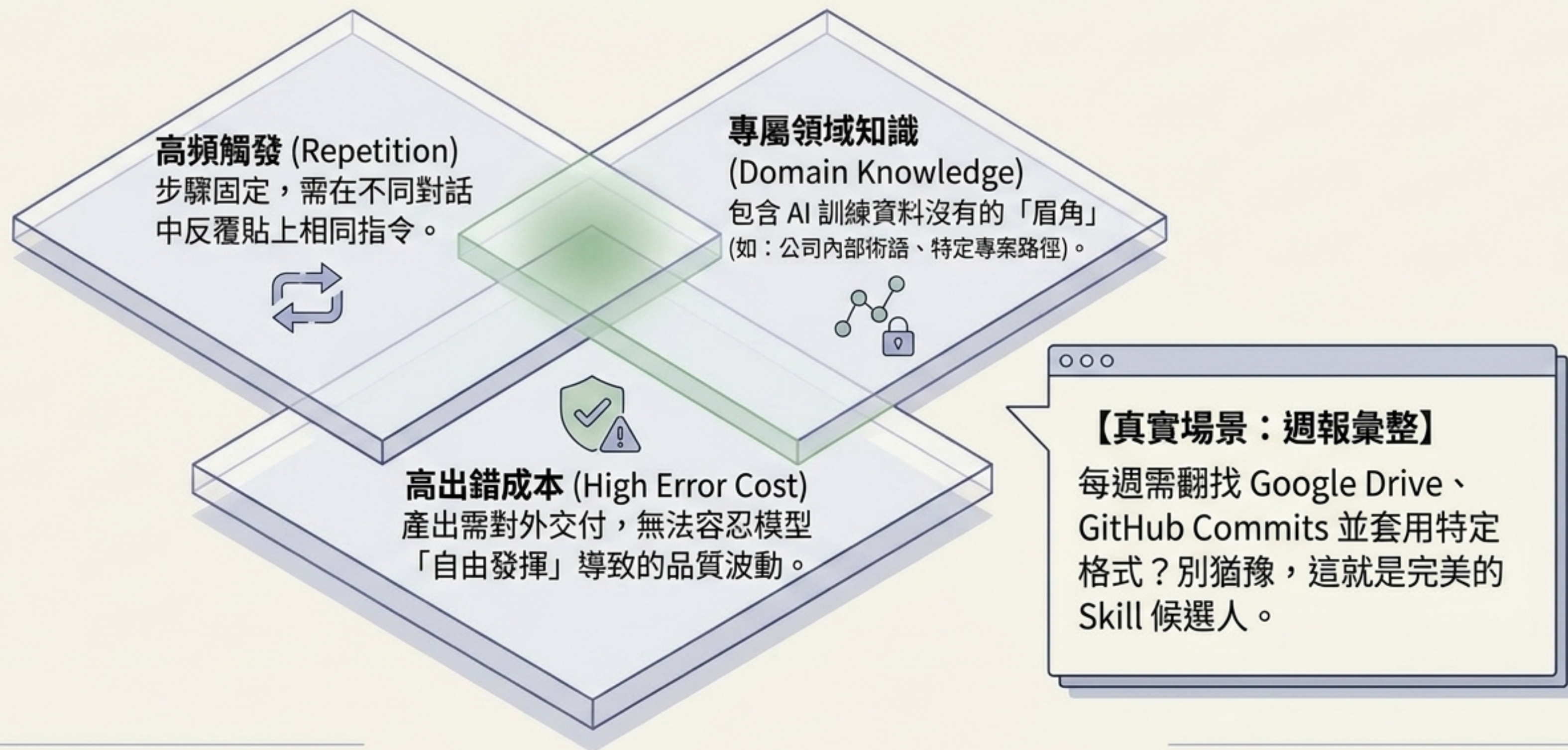
產出穩定

輕鬆維護



Skill 的本質不是一段文字，而是一個「漸進式」的微型軟體系統。

啟動信號：什麼任務值得封裝成 Skill？



架構解析：漸進式披露 (Progressive Disclosure)

Layer 1: Metadata (常駐記憶體)

僅約 100 Tokens。

內容： Name 與 Description。

作用： 讓 Agent 知道「有這個技能」與「何時該喚醒」。

Layer 2: Core Methodology (觸發後載入)

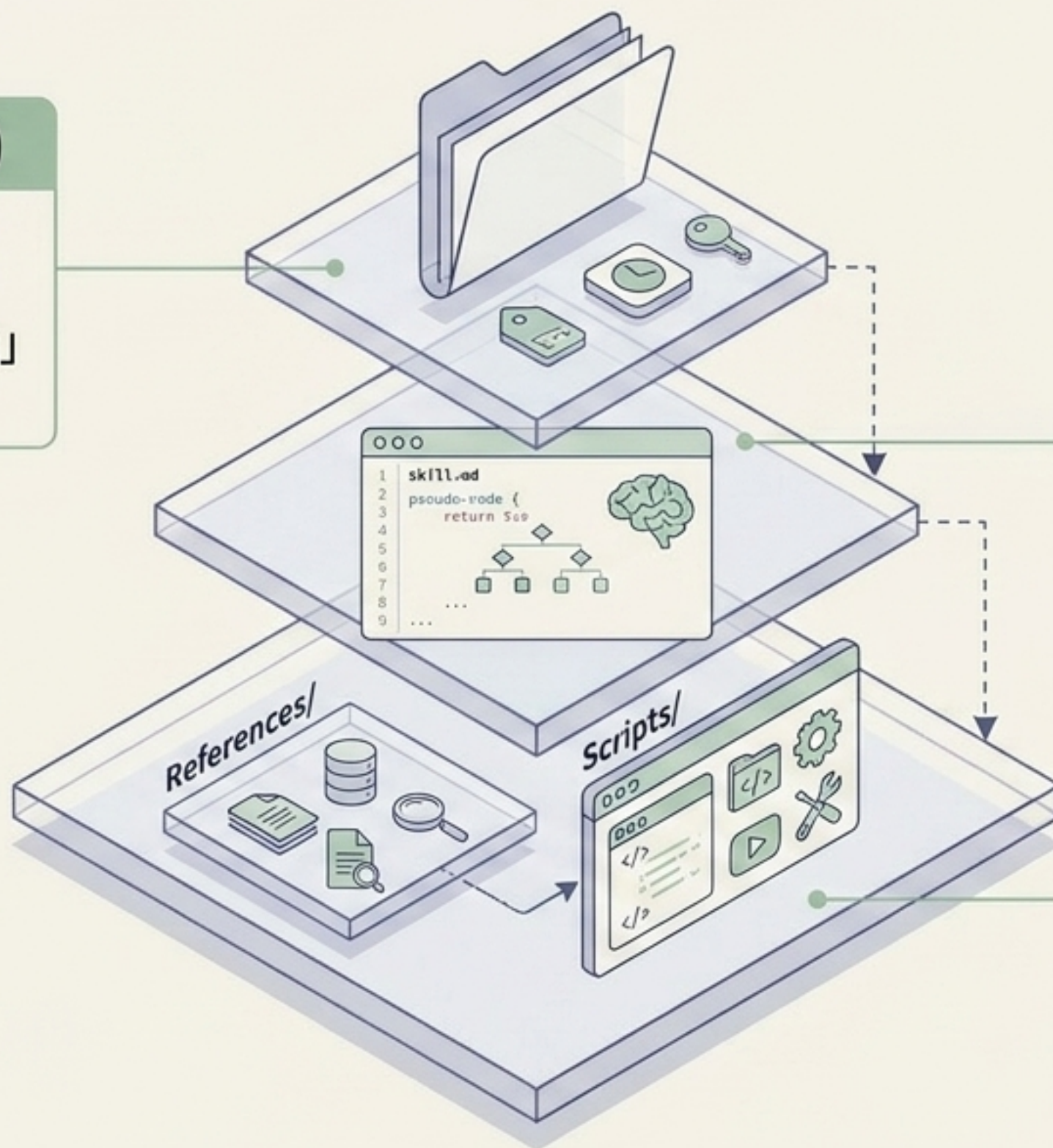
內容： skill.md 主文件。

作用： 傳授 AI 思考邏輯與執行原則（核心心法）。

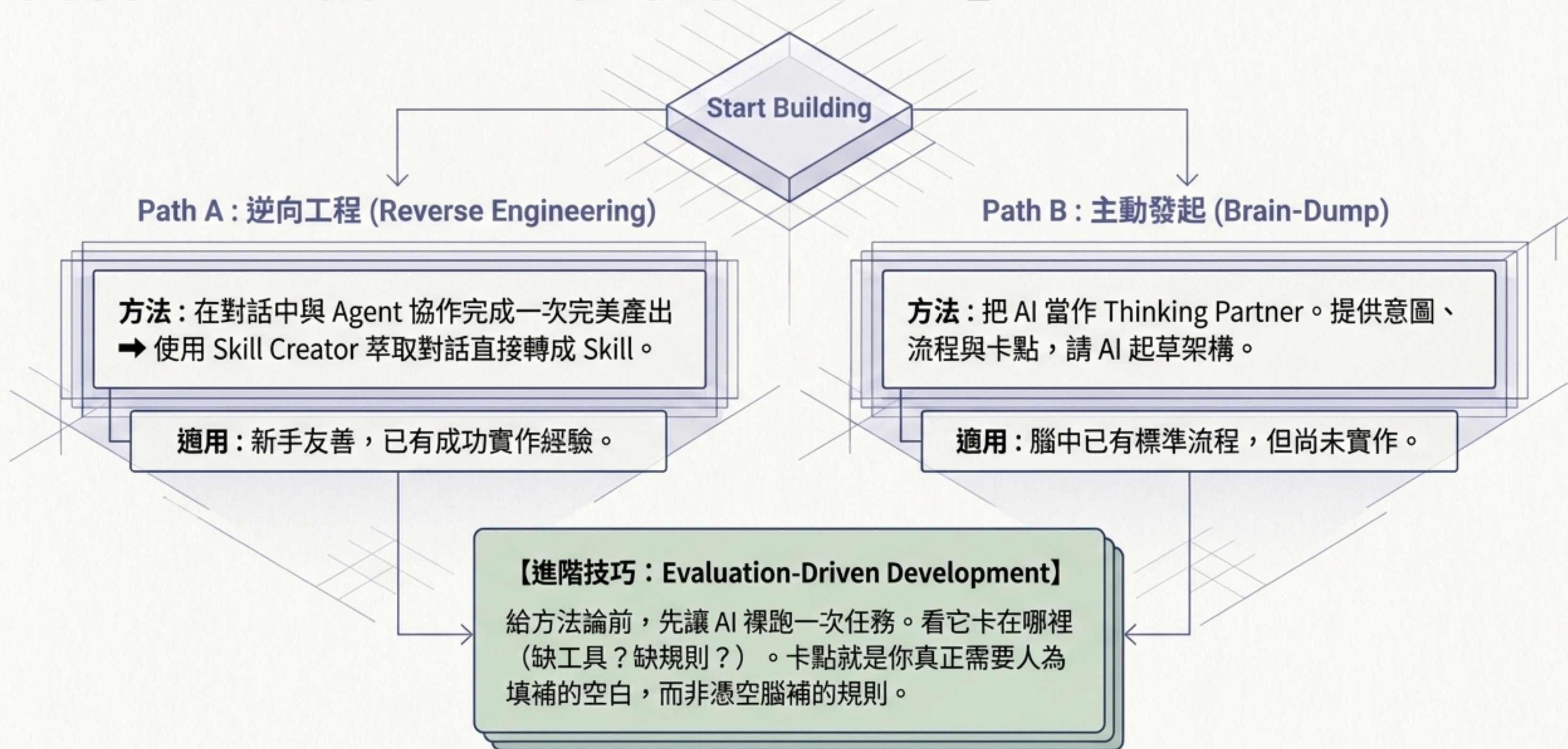
Layer 3: External Assets (按需載入)

內容： References/ (參考資料) & Scripts/ (執行腳本)。

作用： 真正需要時才調用，確保 Context Window 永不塞爆。



兩條打造路徑與「驗收驅動開發」



最佳心法：你不是指令員，你是 AI 的 PM

“「模型推理能力已遠超多數人類，請學會放手。
給予『執行心法』，而非『死板步驟』。」

Why (為什麼這樣做?)

- 賦予任務動機與商業脈絡。

Attitude (抱持什麼心態?)

- 設定篩選資訊的視角與優先級。

Guiding Principles (核心原則)

- 遇到模糊地帶時的判斷依據 (如：遇到特例該怎麼辦)。

不要問 AI 能為你做什麼，問自己有沒有把上下文與目標準備到足以讓 AI 成功交付。

實戰光譜：何時寫「步驟」，何時寫「原則」？

維度	嚴格步驟 (Rigid SOP)	指導原則 (Guiding Principles)
(適用場景)	對錯有「絕對客觀標準」的任務	需要「主觀判斷與彈性」的任務
(舉例)	表單欄位驗證、API 呼叫格式	專案週報彙整、重點摘要
(寫法特徵)	Step 1, Step 2, If X then Y	哲學、原則、優先順序

Real-World Translation	
✖ 寫死步驟：「列出所有 Blocker，並寫出下一步。」	✔ 核心心法：「撰寫時以『目前卡點及下一步行動』為最高優先，無 Blocker 則自行判斷不寫；撰寫標準為『主管需要知道這件事嗎？』」

真實場景一：Agent 不觸發 Skill 怎麼辦？

AI 經常無視 Skill 開始 Freestyle，問題出在 Metadata 給的線索不足。

Bug vs. Fix

Bad Metadata

name: 'do_something'

description: 'This function does a thing.'



Perfect Metadata

name: drafting_weekly_reports

description: Drafts weekly status updates... use when user mentions 'weekly reports', 'team updates', or 'weekly reviews'.

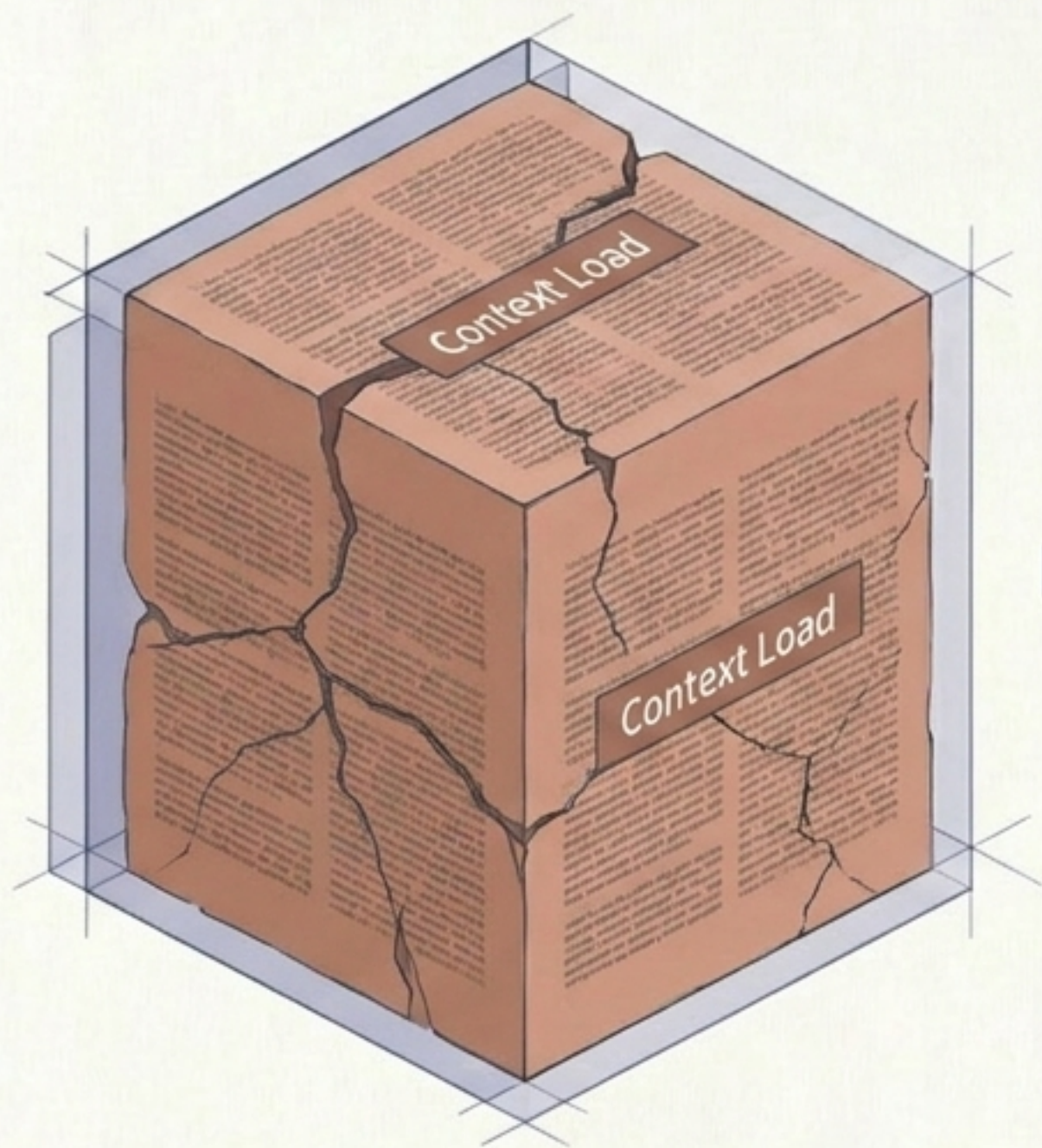
命名法則：[動名詞] + [名詞] → 人機皆可秒懂。

做什麼 + 何時用 (缺一不可)。且絕對用**第三人稱**。

埋入**自然觸發詞** (包含使用者平時說話的自然詞彙)。

真實場景二：文件越寫越長，產出越來越不穩

實戰建議：主文件壓在 200 行內，超過即外包拆分。



References (外掛細節與變體)

- 放什麼：100 行以上的 Output Format、公司內部術語表、極端狀況範例。
- 用法：『請參考 references 內的術語表確保用字相符。』

Scripts (外包確定性操作)

- 放什麼：撈取 GitHub Commits、API 呼叫、排序驗證。
- 用法：讓程式碼處理客觀邏輯，只把結果傳給 AI，消除 Freestyle 的機率。

真實場景三：容錯率極低的商業場景如何防呆？

引入 Sub-agent 控管品質 (Separation of Concerns)。徹底避免主 Agent 身兼球員與裁判，產生幻覺或自我包庇。

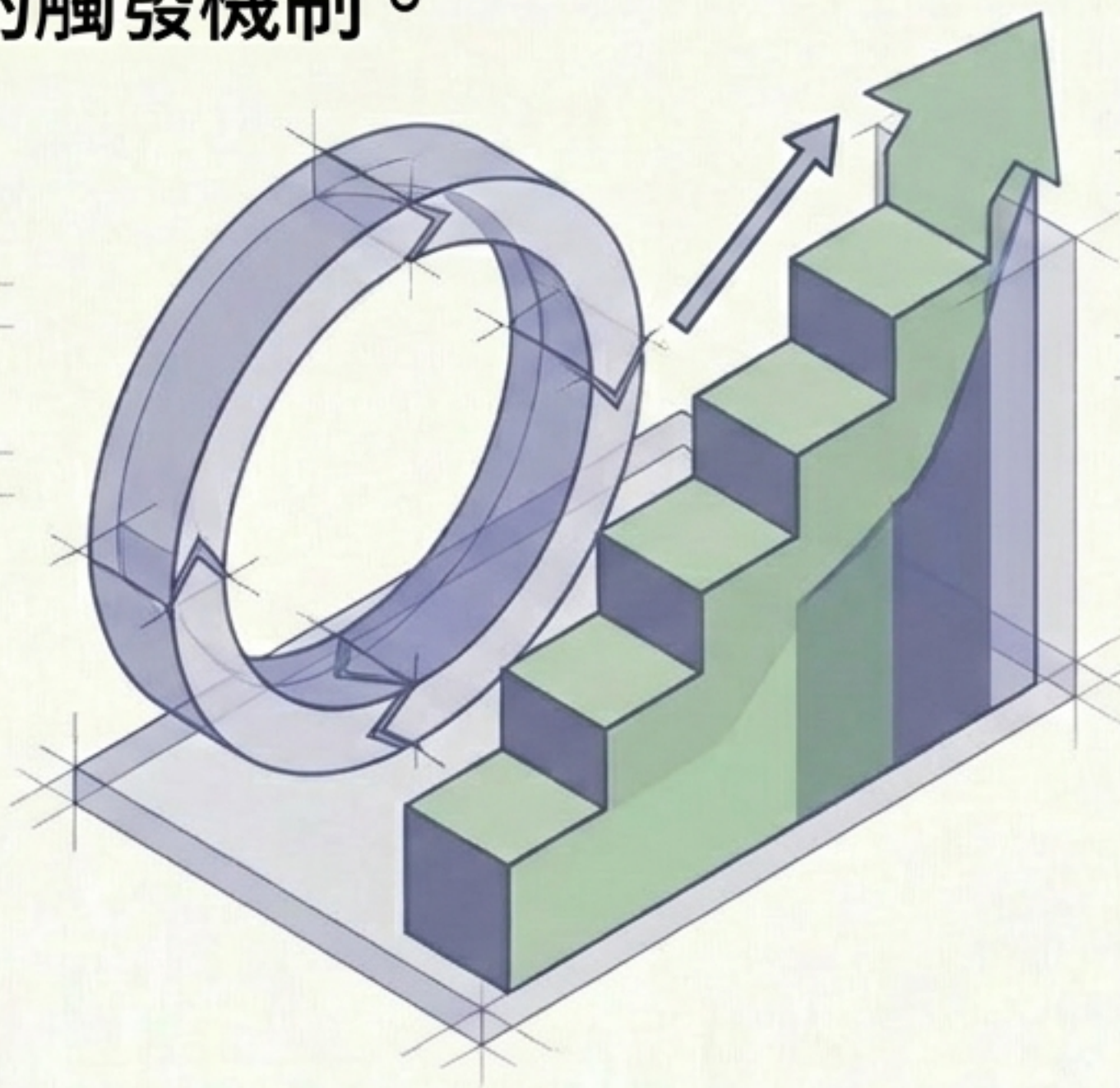


長效維護：從「消耗品」到「數位資產」

放任不管的 Skill 會因冗餘資訊膨脹，變成產出崩壞的元兇。
必須建立何時該維護的觸發機制。

1. 模型升級時的「大掃除」

新模型能力變強後，果斷刪除為了彌補舊模型弱點而加的「補丁規則」。

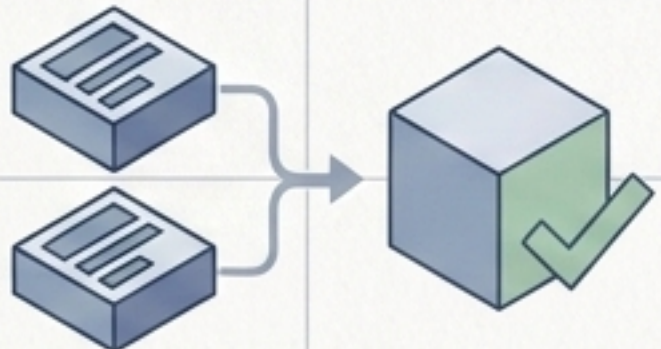
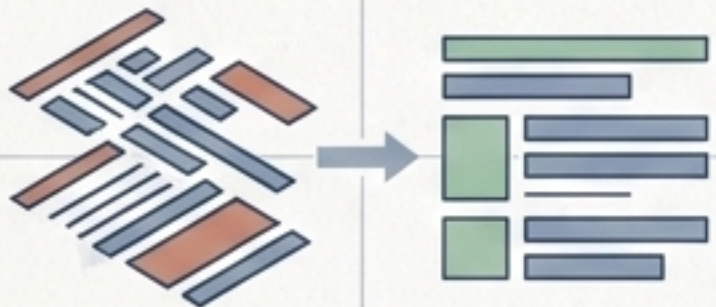
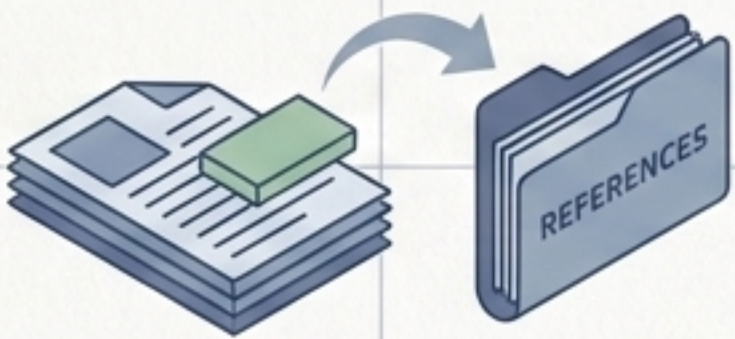


2. workflow 跑完的「自動復盤」

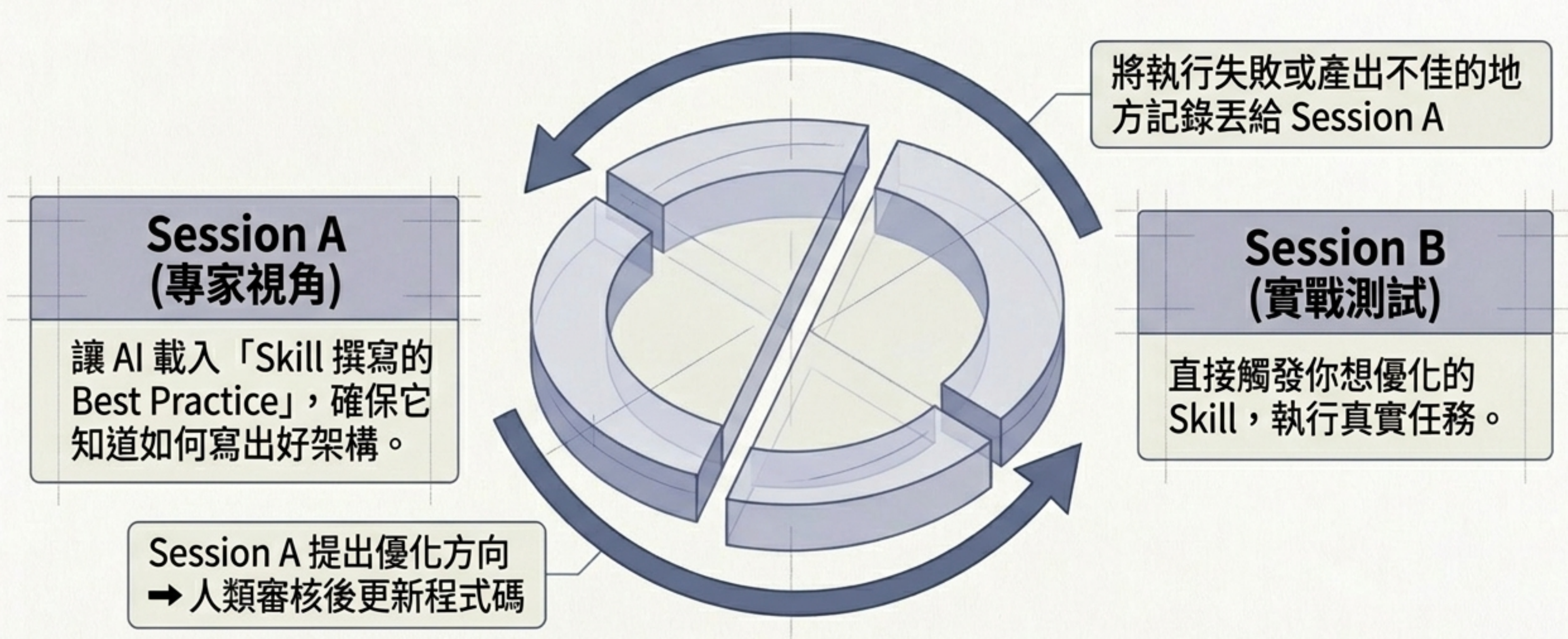
請復盤剛才的執行過程，列出錯誤原因，評估是否需寫入 Skill 避免再犯。

⚠ 陷阱警告：絕對不要讓 AI 完全主導寫入。人類必須擔任 Gatekeeper，審核並確保結構乾淨。

維護實作：每次優化必做的三件事

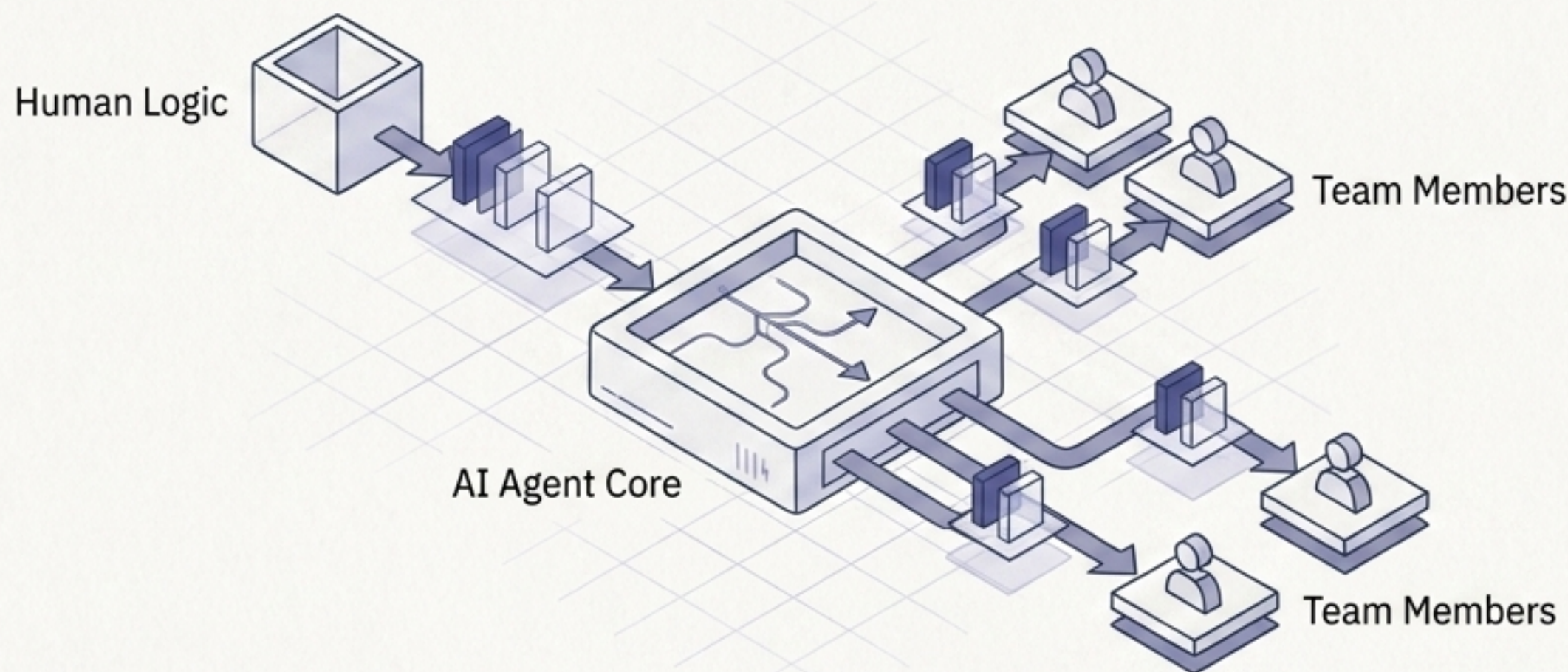
1	刪除冗餘 (Deduplicate) - 同樣的原則開頭講過，中間絕不重複。一件事講一次就好，除非 AI 執行時真的常漏掉。	
2	重整結構 (Restructure) - 快速重讀整份 Skill，如果人類難以一眼看懂骨架，就代表必須重構。無法理解的東西絕對無法維護。	
3	抽取例外 (Extract to References) - 如果主文件裡塞了太多特定情境 (Corner Cases) 才出現的範例，強制將其抽離到 References 文件夾降低維護難度。	

終極優化技巧：雙 Session 測試法



效益：快速且精準地找出日常使用時的惱人地雷，無痛迭代。

真正價值：Harness Engineering 與知識轉移



過去與現在對比

過去：Know-how 鎖在個人腦袋裡，傳遞只能靠 SOP 文件與手把手帶新人。

現在：將累積多年的**方法論**、**判斷標準**與**踩坑經驗**打包成 Skill。整個團隊的 Agent 自動繼承這套做事邏輯。

核心洞察 (Harness Engineering)

寫 Skill 不是為了把 AI 拉高到你的水準，而是把你的「**判斷力與思考邏輯**」轉移給 **Agent**。這是團隊能力共享的全新介面。

AI Skill 實戰架構清單 (一鍵截圖)



📦 漸進式結構 (Architecture)

- 100 Token 完美觸發 (動名詞+名詞 / 做什麼+何時用)。
- 主文件留給核心心法。
- 腳本與參考按需載入。



🧠 核心心法 (Mindset)

- 當 PM 而非指令員。
- 客觀任務給嚴格步驟；主觀判斷給「Why、心態、原則」。



🛡️ 防呆避坑 (Pitfalls)

- 嚴守 200 行原則，超過即拆分。
- 不確定性/品管丟給 Sub-agent。
- 客觀確定性操作丟給 Scripts。



🔄 長效維護 (Maintenance)

- 驗收驅動開發 (裸跑找缺口)。
- 模型升級必清舊補丁。
- 自動復盤由 AI 建議，人類嚴格把關。

記住：最有獨特價值的，永遠是你腦中的「執行心法」。開始封裝你的第一個 Skill 吧！